



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

UNIVERSITI TEKNOLOGI MALAYSIA

Semester II 2025/2026

SECJ 3483 : WEB TECHNOLOGY

DHL DAC 3.0 CHALLENGE & INDUSTRY COLLABORATION

WEB APPLICATION & ROBOTIC PROCESS AUTOMATION (RPA)

SCENARIO : SCENARIO 2

Lecturer's Name: Dr Harry Aiman

Name : Muhammad Azim Bin Adanan

Matric Number : A23MJ5081

Section : 15

1. Introduction and Scenario Description

1.1 Background

DHL Asia Pacific Shared Services (DHL APSSC) is one of the world leading logistics companies handling millions of shipments across the Asia Pacific region every day. The Customer Support division is responsible for receiving, processing, and resolving incident reports from customers and warehouse teams.

This project is developed as part of the DHL Digital Automation Challenge 3.0 (DAC 3.0), an industry collaboration between DHL APSSC and Universiti Teknologi Malaysia. The objective is to build an AI-powered Incident Reporting System that automates the full process of collecting, understanding, assigning, and tracking incident reports.

1.2 Problem Statement

DHL Customer Support teams receive incident reports daily from multiple inconsistent channels:

- Email inboxes containing long complaint threads from customers
- Telegram and Microsoft Teams chat messages from field staff
- Phone call notes written manually by support agents
- Images and screenshots of damaged packages
- Handwritten instructions from warehouse teams

Because information is unstructured and inconsistent, DHL cannot:

- Quickly identify the real issue from raw messy text input
- Automatically assign incidents to the correct department
- Prioritize incidents based on urgency and business impact
- Track the resolution progress of each incident consistently
- Produce consolidated analytics reports for management

This leads to slow response times, repeated manual work, and inconsistent customer service quality.

1.3 Project Objective

This project aims to design and develop an AI-powered Incident Reporting System with three core components:

- A React-based web application for managing and tracking incidents with full CRUD operations
- An offline AI classification engine that automatically categorizes and summarizes incidents
- A UiPath RPA bot that automates file ingestion and sends data to the web application

1.4 Incident Categories

- Late Delivery - parcels not delivered by the expected date
- Damaged Parcel - parcels received in damaged or broken condition
- Address Issue - wrong delivery address or failed address update requests
- System Error - portal, tracking system, or technical failures
- Customer Complaint - general service quality and staff conduct complaints

2. System Architecture

2.1 Overview

The system is designed as a three-tier web application. The frontend is a React single-page application, the backend is a Node.js Express REST API, and the data layer uses lowdb which stores all data in a JSON file. An offline AI engine and a UiPath RPA bot are integrated as additional automation components.

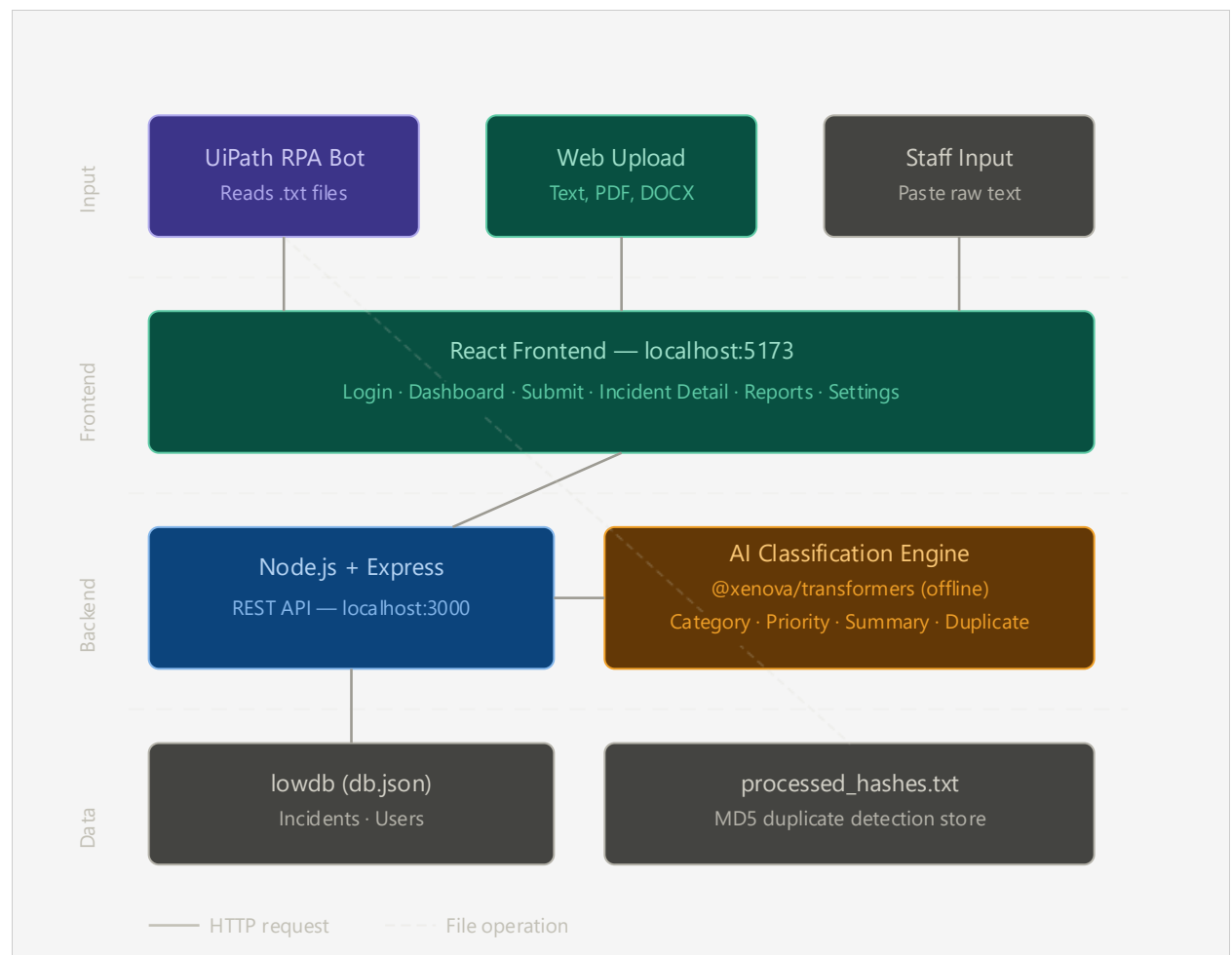


Figure: System Architecture Diagram

2.2 Component Description

Frontend uses React 19 with Vite as the build tool. React Router v7 handles client-side navigation and Axios handles all HTTP API calls. The application runs on port 5173.

Backend is built with Node.js and Express. It handles JWT authentication, all CRUD operations, AI classification triggers, and user management. The backend runs on port 3000.

Database uses lowdb, a lightweight JSON file database stored in db.json. This satisfies Assignment Option 1 which requires JSON format for data storage with full CRUD operations via REST API.

AI Engine uses the @xenova/transformers package with Xenova/distilbart-mnli-12-3 for zero-shot text classification. The model runs completely offline with no external API required.

UiPath RPA Bot is the DHL_IngestBot workflow that automates file ingestion, performs duplicate detection, submits to the API, and generates logs.

2.3 Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + Vite + Axios	User interface and API calls
Routing	React Router v7	Client-side navigation
Backend	Node.js + Express 5	REST API and business logic
Database	lowdb JSON file	Data storage Option 1
Auth	JWT + bcryptjs	Secure login
AI Model	@xenova/transformers	Offline classification
RPA	UiPath Studio 2026	Automated ingestion bot

3. API and JSON Structure

3.1 REST API Endpoints

The system implements a full RESTful API. All endpoints except login and UiPath ingest require a valid JWT Bearer token.

Method	Endpoint	Auth	Description
POST	/api/login	None	Authenticate user returns JWT token
GET	/api/incidents	Bearer	List all incidents with filters
GET	/api/incidents/:id	Bearer	Get single incident
POST	/api/incidents	Bearer	Create incident with AI analysis
PATCH	/api/incidents/:id/status	Bearer	Update incident status
GET	/api/reports	Bearer	Get analytics statistics
POST	/api/ingest	None	UiPath bot endpoint no auth

3.2 JSON Data Structure

Each incident stored in db.json contains the following fields:

Field	Type	Description
id	Number	Unique timestamp-based identifier
raw_content	String	Original unprocessed incident text
title	String	AI-generated 5 to 8 word title
summary	String	AI-generated 2 to 3 sentence clean summary
category	String	AI-classified incident type
priority	String	AI-assigned High Medium or Low
is_duplicate	Boolean	True if similar incident already exists
status	String	Open In Progress or Resolved
source	String	Manual or UiPath Bot
created_at	String	ISO 8601 creation timestamp
updated_at	String	ISO 8601 last update timestamp

4. User Interface Screenshots

4.1 Login Page

The login page provides secure JWT-based authentication. Only registered users can access the system. The default administrator account uses admin@dhl.com with password admin123.

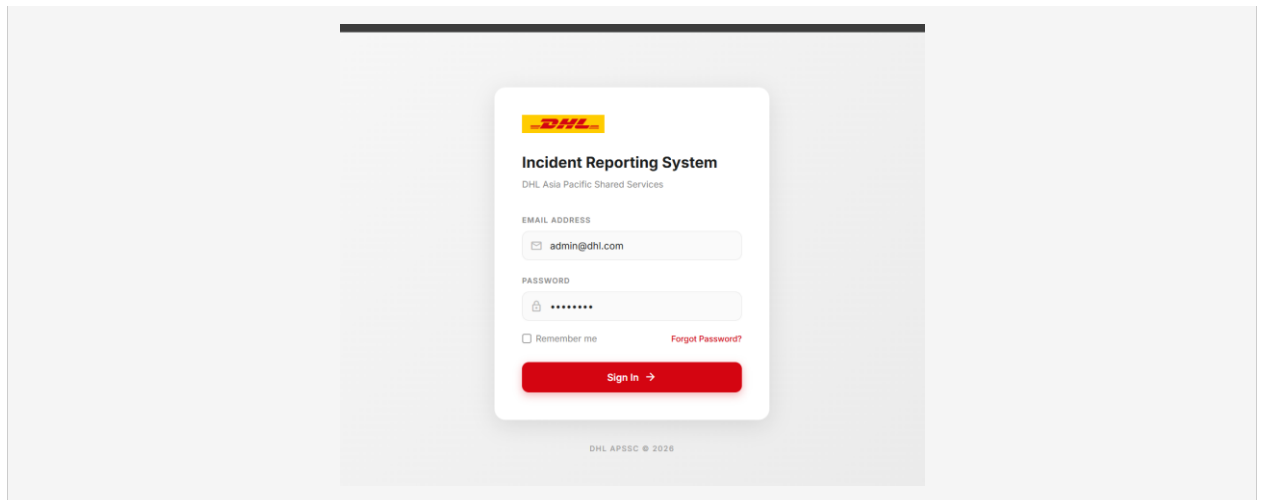


Figure: Login Page

4.2 Dashboard

The main dashboard displays all incidents in a searchable and filterable table. Four summary cards show counts for total open in progress and resolved incidents.

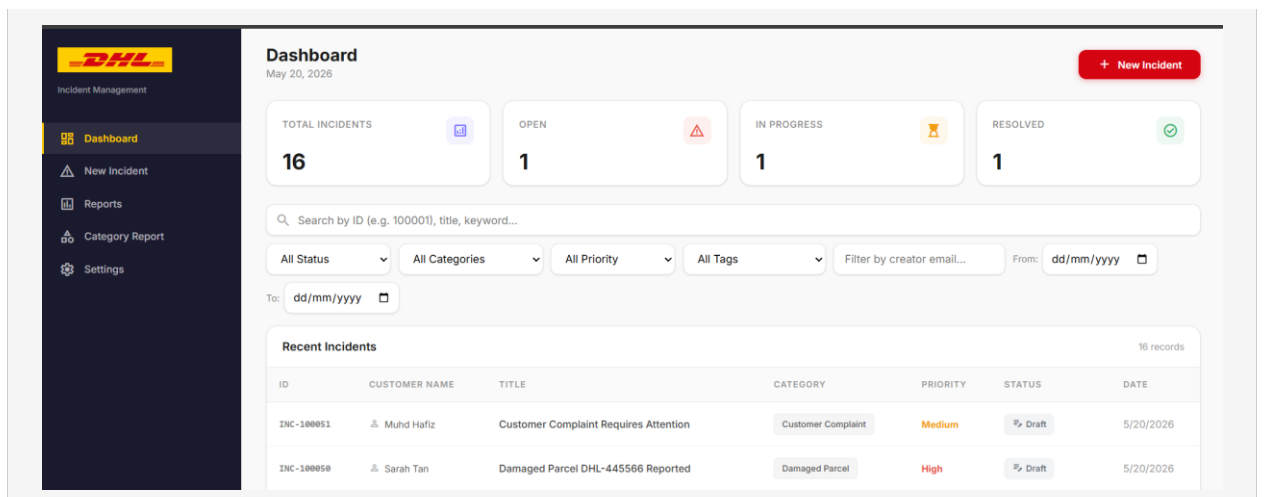


Figure: Dashboard Incident Management Console

4.3 Submit Incident Page

Staff paste raw unstructured text from any source. The AI engine automatically generates the title summary category and priority after submission.

DHL

Incident Management

Dashboard

New Incident

Reports

Category Report

Settings

Dashboard > New Incident

New Incident

Submit a raw incident report for AI-powered classification

Back to Dashboard

Raw Incident Report

UPLOAD FILE (PDF, DOCX, TXT)
Choose File

File content will be extracted and placed in the text area below

INCIDENT DESCRIPTION

Customer email dated 5 May requesting urgent address change for shipment MY44567789. Original address was residential in Shah Alam, new address communicated via email is customer's office in PJ. Email subject reads: "PLEASE UPDATE DELIVERY ADDRESS ASAP".
System shows address update was never applied. Parcel delivered to old address on 6 May and signed by unknown person. WhatsApp message from recipient says, "I already told agent yesterday to change address, why still sent there???"
Phone log from support agent indicates a call on 5 May where agent noted "address change requested - escalate to ops". No ticket was created.
Warehouse handwritten note: "Address updated verbally?? Not in system."
Customer now claims parcel missing and DHL responsible. Customer also asks why email was ignored. Data

Tips for Best Results

- Include specific dates, times, and locations
- Add tracking numbers or order references
- Describe the customer's issue clearly
- Mention any previous interactions
- Note the urgency level if applicable

AI will automatically extract: title, summary, category, priority, and duplicate detection.

Figure: Submit Incident Page with AI Analysis

4.4 Incident Detail Page

Shows the AI-generated summary alongside the original raw input. Staff update the status using workflow buttons. A warning appears when a duplicate is detected.

DHL

Incident Management

Dashboard

New Incident

Reports

Category Report

Settings

Logout

Tracking number MY44567789 already exists in incident #100031

AI Analysis Summary

Customer email dated 5 May requesting urgent address change for shipment MY44567789. Original address was residential in Shah Alam, new address communicated via email is customer's office in PJ.
REFERENCE(S): MY44567789
Supervisor Requested
CUSTOMER DEMANDS:

- Requesting urgent address change for shipment MY44567789
- Escalate to ops

Raw Incident Payload

Customer email dated 5 May requesting urgent address change for shipment MY44567789. Original address was residential in Shah Alam, new address communicated via email is customer's office in PJ. Email subject reads: "PLEASE UPDATE DELIVERY ADDRESS ASAP".
System shows address update was never applied. Parcel delivered to old address on 6 May and signed by unknown person. WhatsApp message from recipient says, "I already told agent yesterday to change address, why still sent there???"
Phone log from support agent indicates a call on 5 May where agent noted "address change requested - escalate to ops". No ticket was created.
Warehouse handwritten note: "Address updated verbally?? Not in system."
Customer now claims parcel missing and DHL responsible. Customer also asks why email was ignored. Data spread across email, WhatsApp, call notes, and physical handwritten notes.

Incident Details

CUSTOMER NAME: Unknown
CATEGORY: Address Issue
PRIORITY: High
SOURCE: Manual
CREATED: 5/20/2026, 9:50:09 PM
LAST UPDATED: 5/20/2026, 9:50:09 PM
SUBMITTED BY: admin@dhl.com

Status Workflow

VERSIONING

Draft

Reviewed

Figure: Incident Detail Page with AI Summary and Status Workflow

4.5 Reports Page

Provides management-level analytics including total counts category breakdown with bars and priority distribution. All data is fetched live from the API.

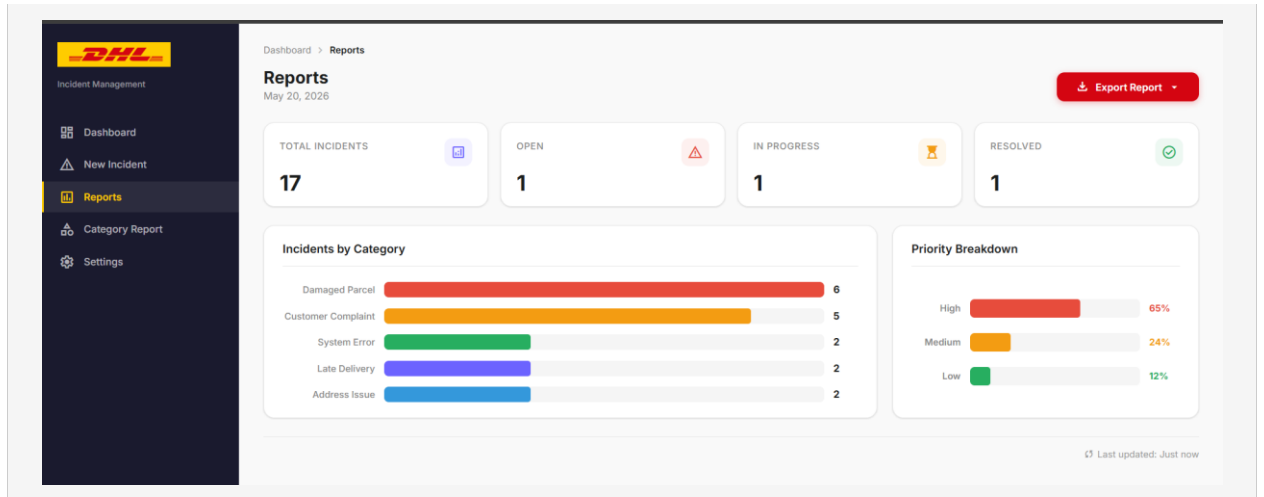


Figure: Reports and Analytics Page

5. RPA Workflow and Automation Explanation

5.1 Overview

The DHL_IngestBot is a UiPath Studio automation workflow that satisfies the mandatory RPA component of this assignment. The bot reads customer complaint files from a local folder, performs duplicate detection, submits each file to the backend API, and generates a summary log after every run.

5.2 Process Justification

The file ingestion process was selected for RPA automation because:

- It is highly repetitive requiring the same steps for every single file processed
- High volume means manual processing would be impractical for DHL daily operations
- Automation ensures consistent duplicate checking and API submission logic
- Every run produces an auditable log file for traceability and compliance

5.3 Workflow Design

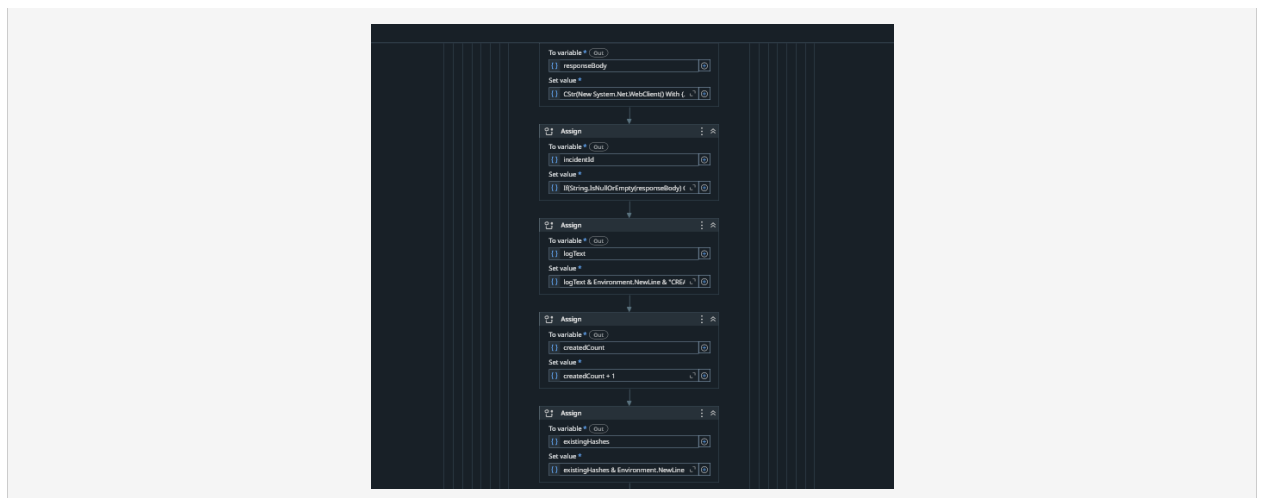


Figure: UiPath Studio Main.xaml Workflow Screenshot

5.4 Step by Step Automation Logic

Step 1 Initialization: Counter variables set to zero. Log text initialized with timestamp. Processed hashes file read into memory.

Step 2 File Discovery: Bot scans incoming folder for all .txt files using System.IO.Directory.GetFiles and processes each file inside a For Each loop.

Step 3 Duplicate Detection: For each file the bot computes an MD5 hash of the content. Hash is compared against processed hashes file. If found the file is skipped as duplicate.

Step 4 API Submission: For new files the bot reads the content and sends an HTTP POST request to <http://localhost:3000/api/ingest> with raw text and source identifier.

Step 5 Response Handling: On HTTP 201 success the bot extracts the incident ID from the JSON response. File is moved to processed folder and hash is saved.

Step 6 Error Handling: All operations wrapped in Try Catch. On failure the file moves to errors folder and the error is logged.

Step 7 Log Generation: After all files processed the bot writes a complete run log to the logs folder with a timestamp filename.

5.5 RPA Integration with Web Application

The UiPath bot connects to the web application through the `/api/ingest` REST endpoint. This endpoint requires no JWT token allowing the bot to submit incidents without a login session. Once submitted the backend AI engine classifies the incident and the new incident appears on the dashboard immediately.

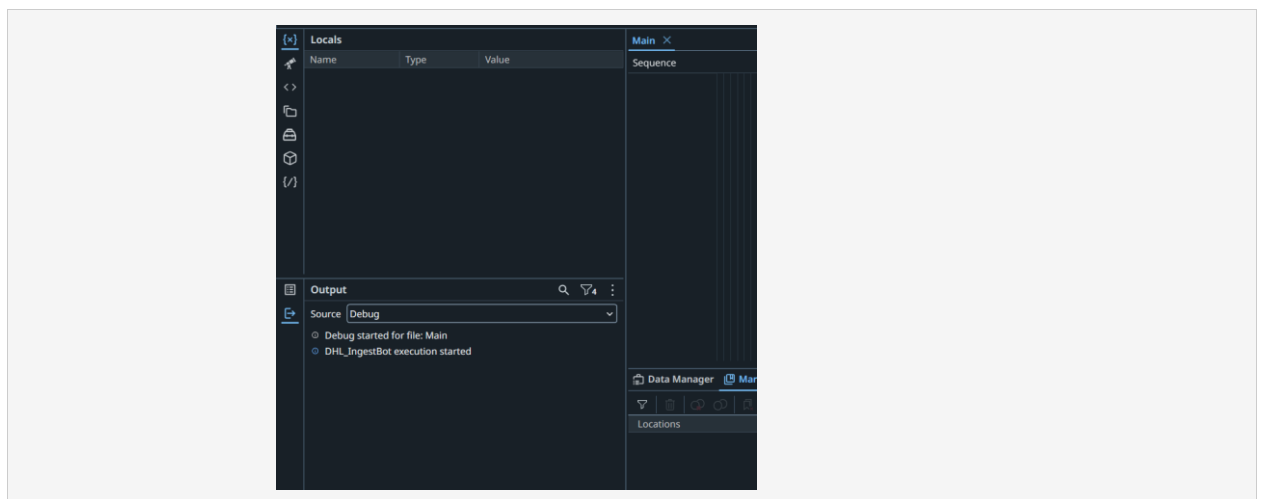


Figure: UiPath Output Panel showing CREATED log

Dashboard > INC-100050

←

Damaged Parcel DHL-445566 Reported

INC-100050 • UiPath Bot • 5/20/2026, 9:42:30 PM

Draft

📄 AI Analysis Summary

Damaged parcel received - DHL-445566

From: sarah.tan@gmail.com • Date: 20 May 2026

I paid RM350 for this item and I am very disappointed with the condition it arrived in. I have taken photos of the damage and I am requesting either a full refund of RM350 or a replacement item to be sent immediately.

🔍 REFERENCE(S): DHL-445566

📈 AMOUNTS MENTIONED: RM350

📄 CUSTOMER DEMANDS:

- Requesting either a full refund of RM350 or a replacement item to be sent immediately

📄 Raw Incident Payload

🔍 Incident Details

CUSTOMER NAME	Sarah Tan
CATEGORY	Damaged Parcel
PRIORITY	High
SOURCE	UiPath Bot
CREATED	5/20/2026, 9:42:30 PM
LAST UPDATED	5/20/2026, 9:42:30 PM

🔗 Status Workflow

Figure: Dashboard showing new incident created by UiPath Bot

6. Conclusion

This project successfully delivers an AI-powered Incident Reporting System that addresses the core operational challenge faced by DHL Customer Support. The system automates the full pipeline from raw messy unstructured input to a classified prioritized tracked and reportable incident record.

Key achievements of this project:

- A fully functional React web application with secure JWT authentication complete CRUD operations via REST API real-time search and filtering and clear status workflow
- An offline AI classification engine that categorizes incidents assigns priority generates clean summaries and detects duplicates
- A UiPath RPA bot that automates file ingestion with duplicate detection error handling and automated log generation
- Full integration between UiPath bot and web application through the /api/ingest REST endpoint

The system solves the DHL problem of slow manual processing by eliminating repetitive data entry and providing management with real-time analytics.

References

DHL APSSC. (2026). DAC 3.0 Challenge Booklet. DHL Asia Pacific Shared Services.

React. (2024). React documentation. <https://react.dev>

UiPath. (2024). UiPath Studio user guide. <https://docs.uipath.com/studio>

Xenova. (2024). @xenova/transformers. <https://github.com/xenova/transformers.js>

Express.js. (2024). Express web framework. <https://expressjs.com>